

DESIGN AND IMPLEMENTATION OF PASSWORD MANAGER

A Project report submitted
in partial fulfillment of requirement for the award of degree

BACHELOR OF TECHNOLOGY

in

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

by

CHIDIRALA RAKSHITH

2203A51276

CHITLA JAYADEEP

2203A51277

Under the guidance of

Dr. Santosh Kumar

Associate Professor, School of CS&AI.



SR University, Ananthasagar, Warangal, Telangana-506371

CERTIFICATE

This is to certify that this project entitled “**DESIGN AND IMPLEMENTATION OF PASSWORD MANAGER**” is the bonafied work carried out by **RAKSHITH, JAYADEEP**.

As a Project for the partial fulfilment to award the degree **BACHELOR OF TECHNOLOGY** in **School of Computer Science and Artificial Intelligence** during the academic year

2024-2025 under our guidance and Supervision.

Dr. Santosh Kumar .H

Associate Professor

SR University

Anathasagar, Warangal

Dr. M. Sheshikala

Professor & Head,

School of CS&AI,

SR University

Ananthasagar, Warangal.

ACKNOWLEDGEMENT

We owe an enormous debt of gratitude to our Major Project guide **Dr. Santosh Kumar, Assoc. Prof** as well as Head of the School of CS&AI, **Dr. M. Sheshikala, Professor** and Dean of the School of CS&AI, **Dr. Indrajeet Gupta Professor** for guiding us from the beginning through the end of the Capstone Project with their intellectual advices and insightful suggestions. We truly value their consistent feedback on our progress, which was always constructive and encouraging and ultimately drove us to the right direction.

Finally, we express our thanks to all the teaching and non-teaching staff of the department for their suggestions and timely support.

CONTENTS

S.NO.	TITLE	PAGE NO.
1	INTRODUCTION	1
2	PROBLEM IDENTIFICATATION	2
3	REQUIREMENT ANALYSIS	3
4	PROPOSED SOLUTION	4
5	ARCHITECTURE DIAGRAM	5
6	FLOW CHART	6
7	IMPEMENTATION	7 - 8
8	PROGRAM	9 - 13
9	RESULTS	14 - 16
10	LEARNING OUTCOME	17 - 18
11	PROJECT IMPACT AND FUTURE SCOPE	19-20
12	CONCLUSION	21
16	REFERENCES	22

LIST OF FIGURES

Fig no	Title	Page no
1	Architecture Diagram	5
2	Flow Chart	6
3.	Result screens	14-16

ABSTRACT

In the digital age, managing a growing number of online accounts securely has become increasingly challenging for users. Weak or reused passwords are common security risks that can lead to data breaches and identity theft. This project presents the design and implementation of a Password Manager, a software application aimed at securely storing, managing, and retrieving user credentials.

The password manager enables users to store login credentials in an encrypted format, ensuring data confidentiality through modern cryptographic techniques. The system is protected by a master password, which serves as the sole access key to the encrypted data. It includes functionalities such as adding, retrieving, deleting, and listing passwords, along with import/export options for backups. A GUI-based version enhances usability for non-technical users, while a command-line version ensures cross-platform compatibility.

The implementation emphasizes security through the use of hashing algorithms like SHA-256 for password verification and symmetric encryption (Fernet from the Cryptography library) for data storage. Additionally, password strength checks are incorporated to encourage users to adopt stronger security practices.

This project demonstrates how simple yet effective security tools can help users manage their digital identities more safely, promoting better cybersecurity hygiene in daily life.

INTRODUCTION

In today's digital landscape, individuals rely heavily on numerous online platforms for banking, shopping, education, communication, and more. Each of these services requires authentication—often in the form of a username and password. As the number of accounts increases, so does the challenge of managing multiple unique and secure passwords. Users frequently resort to unsafe practices such as reusing passwords or writing them down, which poses a significant threat to personal data security.

A Password Manager is a secure application designed to store and manage users' credentials in an encrypted format, accessible through a single master password. It helps users maintain strong, unique passwords for each account without the need to memorize them all. The master password serves as the only key needed to unlock access to the stored data.

This project focuses on the design and implementation of a simple yet secure password manager using Python. The application provides core functionalities such as saving new passwords, retrieving stored credentials, deleting outdated entries, and exporting or importing password databases. The application uses cryptographic techniques such as SHA-256 hashing for secure master password validation and symmetric encryption for password confidentiality.

Furthermore, a Graphical User Interface (GUI) is implemented using Tkinter to enhance user interaction and make the tool more accessible to non-technical users. Through this project, we aim to demonstrate how security and usability can be balanced to create an effective solution for password management in everyday life.

PROBLEM IDENTIFICATION

With the increasing reliance on digital services, users are required to create and manage a large number of online accounts. Each service typically demands a unique, strong password to protect against unauthorized access. However, remembering numerous complex passwords is difficult, leading users to adopt poor security practices such as:

- Using weak or easily guessable passwords.
- Reusing the same password across multiple platforms.
- Writing down passwords in insecure locations (e.g., notepads, sticky notes).
- Storing credentials in plain text files on their devices.

These practices significantly increase the risk of data breaches, identity theft, and financial loss. Moreover, the lack of secure, user-friendly tools for password management discourages people from following best practices in cybersecurity.

Additionally, while there are commercial password managers available, many of them are either subscriptionbased, closed-source, or overly complex for average users. This creates a gap for a simple, secure, and open alternative that empowers users to manage their credentials safely and conveniently.

Thus, there is a clear need for a lightweight and easy-to-use password management system that:

- Encourages the use of strong, unique passwords.
- Protects stored data through encryption.
- Offers easy access through a secure master password.
- Provides essential features like add, retrieve, delete, and backup functionalities.

This project addresses these needs by designing and implementing a password manager that ensures both data security and user-friendliness, helping individuals adopt better password management habits in their daily digital lives.

Requirement Analysis, Risk Analysis, Feasibility Analysis

The "Design and Implementation of a Password Manager" project aims to provide users with a secure and user-friendly tool to manage multiple credentials safely. In today's digital age, people rely on various services that require secure login credentials. Without proper management, users often adopt unsafe practices, such as reusing or writing down passwords. This project addresses this issue through a secure and lightweight password manager.

Requirement Analysis

The application must include core functionalities such as storing, retrieving, and deleting passwords, all protected by a master password. It should offer password strength checks to encourage good security habits and support encrypted storage using techniques like SHA-256 hashing and Fernet encryption. From a usability standpoint, the application will offer both command-line and graphical user interface (GUI) options using Tkinter to cater to both technical and non-technical users. Non-functional requirements emphasize the importance of security, performance, portability, and maintainability, ensuring a smooth user experience.

Risk Analysis

The project identifies potential risks such as users creating weak master passwords, forgetting them, or experiencing data corruption. Security breaches may also occur if the password file is accessed by an unauthorized party. These risks are mitigated by enforcing password strength checks, clearly informing users about the importance of safeguarding their master password, and offering backup/restore functionality.

Utilizing trusted libraries like Python's cryptography ensures robust encryption to minimize risks.

Feasibility Analysis

The project is technically feasible using Python and standard libraries. Economically, it is cost-effective as it uses open-source tools, requiring no licensing. Operational feasibility is high since the system is designed to be intuitive and easy to operate, especially with the GUI component. Legally, there are no concerns since all technologies used are free and open-source, and the application does not collect or transmit personal data. In summary, the password manager project is practical, secure, and relevant in addressing real-world password management challenges, offering a balanced solution that emphasizes security, simplicity, and accessibility.

PROPOSED SOLUTION

The proposed solution is to develop a lightweight, secure, and user-friendly **Password Manager** application that allows users to store and manage their login credentials in an encrypted format. The system is designed to eliminate the risks associated with weak, reused, or forgotten passwords by offering a centralized, encrypted storage mechanism protected by a **master password**. **Key Features of the Solution:**

1. Master Password Protection

- The application is accessible only through a strong master password.
- The master password is hashed using **SHA-256** and never stored in plain text.

2. Secure Password Storage

- All account passwords are encrypted using **Fernet symmetric encryption** from the cryptography library.
- Data is stored in a local text file in an unreadable, encrypted format.

3. Password Management

- Users can:
 - Add new credentials (username + password).
 - Retrieve stored passwords.
 - Delete specific account entries.
 - View a list of all stored credentials (if authenticated).

4. Password Strength Validation

- The system encourages strong passwords by validating length and complexity during entry.

5. Import/Export Functionality

- Users can back up encrypted password data to an external file and restore it later.

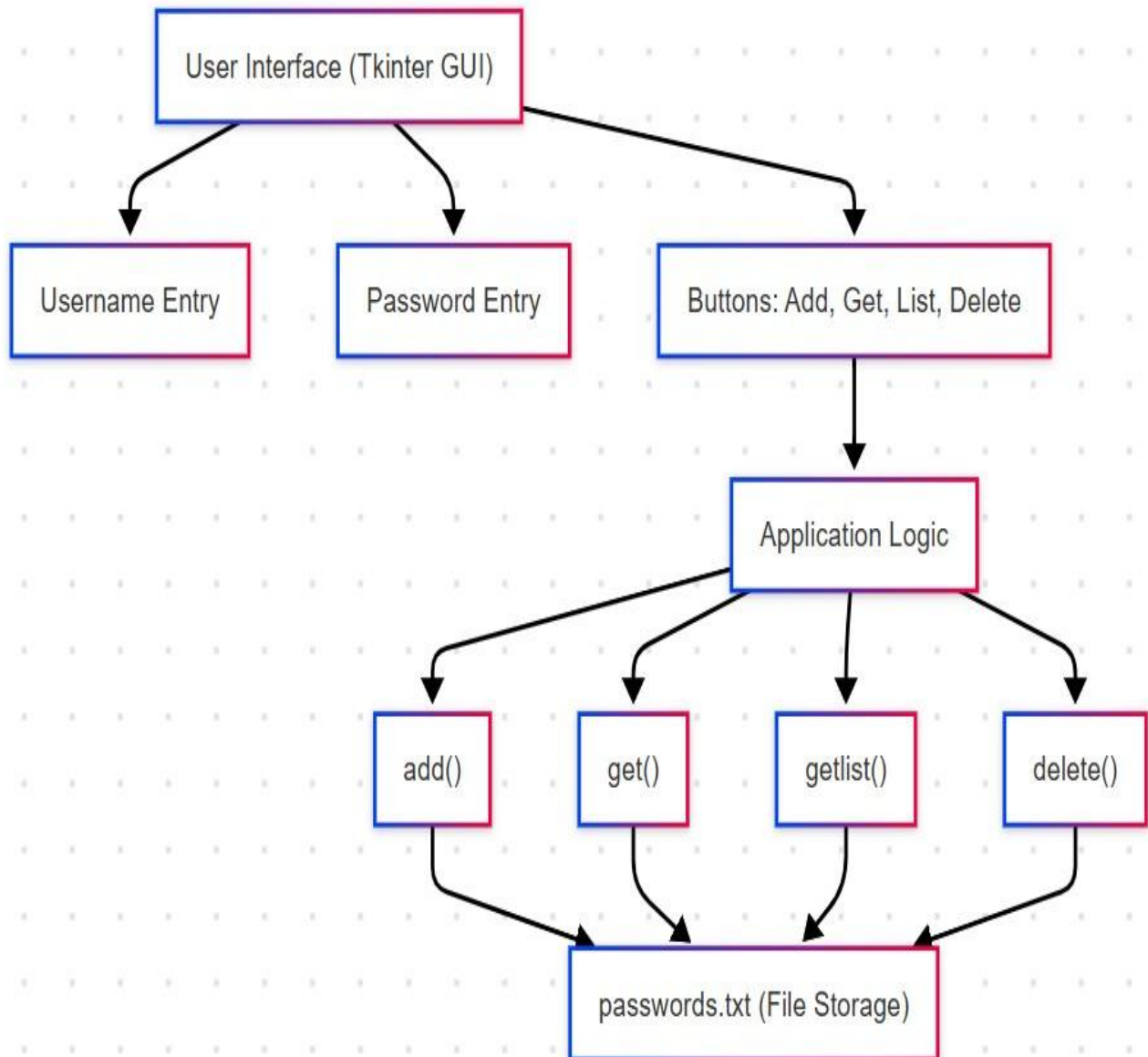
6. Graphical User Interface (GUI)

- The application includes a Tkinter-based GUI for ease of use, especially for non-technical users.

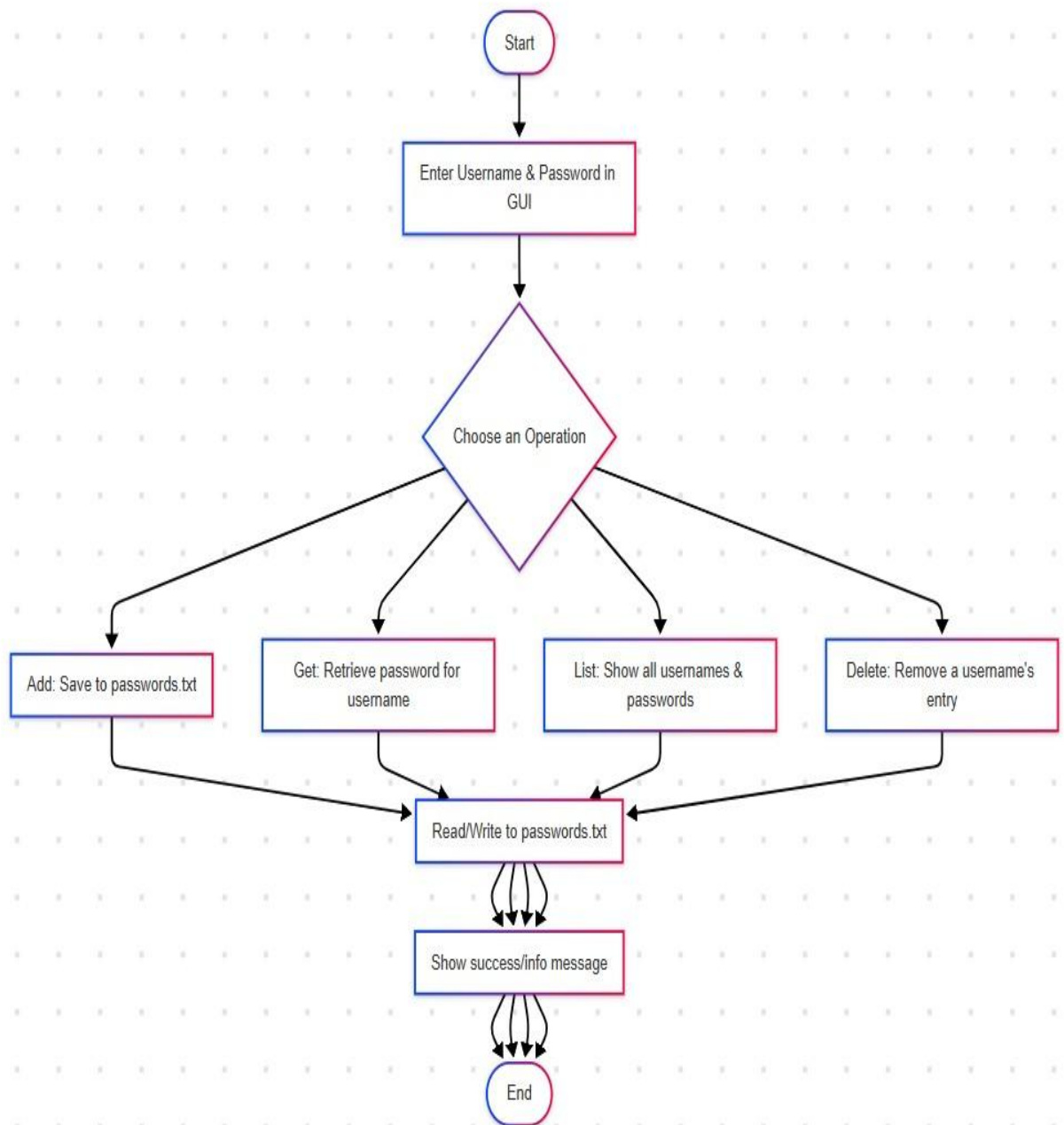
- Users can perform all actions (Add, Get, Delete, List) through button clicks and input fields. **Advantages of the Proposed Solution:**

- Increases password security by removing the need to remember multiple credentials.
- Uses strong encryption to ensure stored data is safe even if accessed by unauthorized individuals.
- Easily portable, requiring only Python and standard libraries.
- Intuitive interface makes it accessible to a wide range of users.

ARCHITECTURE DIAGRAM



FLOW CHART



IMPLEMENTATION

1. Project Objective

To develop a simple GUI-based password manager that allows users to:

- Save usernames and passwords
- Retrieve stored passwords
- List all saved credentials
- Delete specific user credentials

2. Technologies Used

- **Python** – core programming language
- **Tkinter** – for creating the graphical user interface (GUI)
- **Text File Storage** – passwords are stored in a local .txt file
- **MessageBox** – to show user feedback and alerts

3. Architecture Overview

3.1 User Interface

(Frontend) • Built
with Tkinter •

Inputs:

- Username (text entry)
- Password (text entry, masked)
 - Buttons:
 - **Add** – Save new credentials
 - **Get** – Retrieve a password
 - **List** – View all credentials
 - **Delete** – Remove an entry

3.2 Application Logic

(Backend)

- Processes button
click events
 - Handles:
 - Input validation
 - File operations (read, write, search, delete)
 - Error handling ○ Displaying results via pop-up
messages

3.3 File Storage

- Uses a simple text file named passwords.txt
- Each line stores: username password
- Acts as a lightweight, persistent data store

4. Workflow Summary

1. **User launches the app** and enters input

2. **Chooses an action** (Add/Get/List/Delete)
3. **App validates input** and performs the appropriate operation
4. **Data is written to or read from** the text file
5. **User receives feedback** via message box

5. Security Considerations (For Future Enhancement)

- Passwords are stored in plain text — not secure
- Suggested improvements:
- Use encryption or hashing
- Introduce master password access
- Switch to a database like SQLite for better structure

PROGRAM

```
import tkinter as tk from tkinter

import messagebox

def add():    username =

entryName.get().strip()    password =

entryPassword.get().strip()

    if username and password:        with

open("passwords.txt", 'a') as f:

    f.write(f'{username} {password}\n')

messagebox.showinfo("Success", "Password added!")

entryName.delete(0, tk.END)    entryPassword.delete(0,

tk.END)

    else:

        messagebox.showerror("Error", "Please enter both username and password.")

def get():

    username = entryName.get().strip()

if not username:

    messagebox.showerror("Error", "Please enter a username.")

    return
```

```

passwords = {}

try:    with open("passwords.txt",
'r') as f:    for line in f:

        parts = line.strip().split(' ')

if len(parts) >= 2:

    passwords[parts[0]]    =    parts[1]

except FileNotFoundError:

    messagebox.showinfo("Info", "No passwords stored yet.")

    return


if username in passwords:    messagebox.showinfo("Password Found", f"Password for
{username}: {passwords[username]}")

else:

    messagebox.showinfo("Not Found", "No such username exists.")


def getlist():

    passwords = {}

    try:    with open("passwords.txt",
'r') as f:    for line in f:

        parts = line.strip().split(' ')

if len(parts) >= 2:

    passwords[parts[0]]    =    parts[1]

except FileNotFoundError:

    messagebox.showinfo("Info", "No passwords found.")

```



```

    return

    if passwords:

        message = "List of saved passwords:\n"
        for
user, pwd in passwords.items():
            message +=
f'{user}: {pwd}\n'
        messagebox.showinfo("All
Passwords", message)

    else:

        messagebox.showinfo("Empty", "No passwords stored.")


def delete():
    username =
entryName.get().strip()
    if not
username:

        messagebox.showerror("Error", "Please enter a username to delete.")

    return


temp_passwords = []

found = False

    try:
        with open("passwords.txt",
'r') as f:
            for line in f:

                parts = line.strip().split(' ')

    if parts[0] != username:

        temp_passwords.append(line)

    else:

```

```

        found = True

    with open("passwords.txt", 'w') as f:

    for line in temp_passwords:

        f.write(line)

    if found:
        messagebox.showinfo("Deleted", f"User '{username}' deleted
successfully.")

    else:

        messagebox.showinfo("Not Found", "No such user to delete.")

except Exception as e:

    messagebox.showerror("Error", f"An error occurred: {e}")

# GUI Setup if __name__

== "__main__":

    app = tk.Tk()

    app.geometry("400x220")

    app.title("Simple Password Manager")

    # Username    labelName = tk.Label(app, text="Username:")

    labelName.grid(row=0, column=0, padx=10, pady=10, sticky="w")

    entryName = tk.Entry(app, width=30)    entryName.grid(row=0,

    column=1, padx=10, pady=10)

```

```

# Password    labelPassword = tk.Label(app, text="Password:")

labelPassword.grid(row=1, column=0, padx=10, pady=10, sticky="w")

entryPassword = tk.Entry(app, width=30, show="*")

entryPassword.grid(row=1, column=1, padx=10, pady=10)


# Buttons    buttonAdd = tk.Button(app, text="Add", width=15,
command=add)    buttonAdd.grid(row=2, column=0, padx=10,
pady=5)


    buttonGet    =    tk.Button(app,    text="Get",    width=15,    command=get)
buttonGet.grid(row=2, column=1, padx=10, pady=5)


    buttonList    =    tk.Button(app,    text="List",    width=15,    command=getlist)
buttonList.grid(row=3, column=0, padx=10, pady=5)

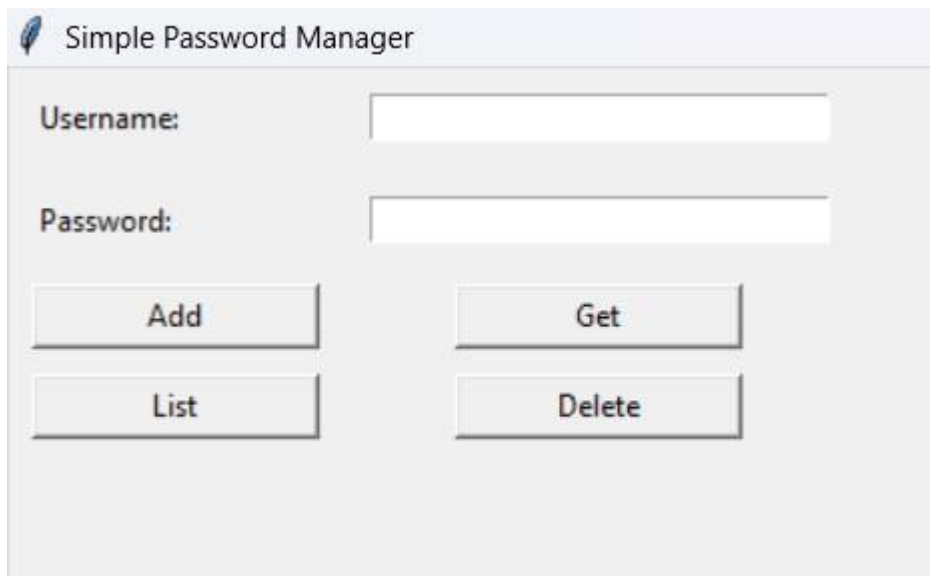

    buttonDelete    =    tk.Button(app,    text="Delete",    width=15,    command=delete)
buttonDelete.grid(row=3, column=1, padx=10, pady=5)


app.mainloop()

```

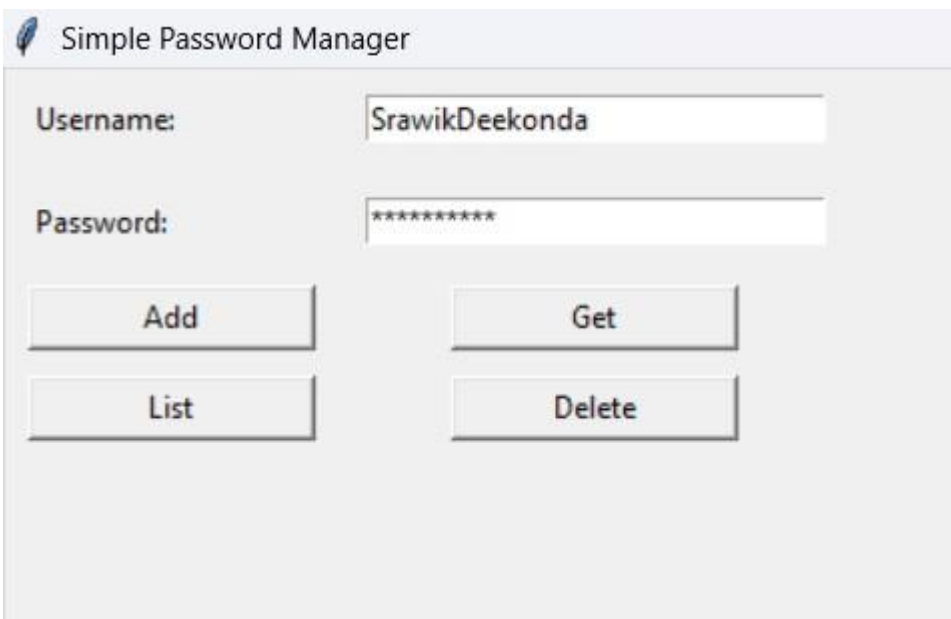
RESULTS

1. Getting GUI (Graphical User Interface)



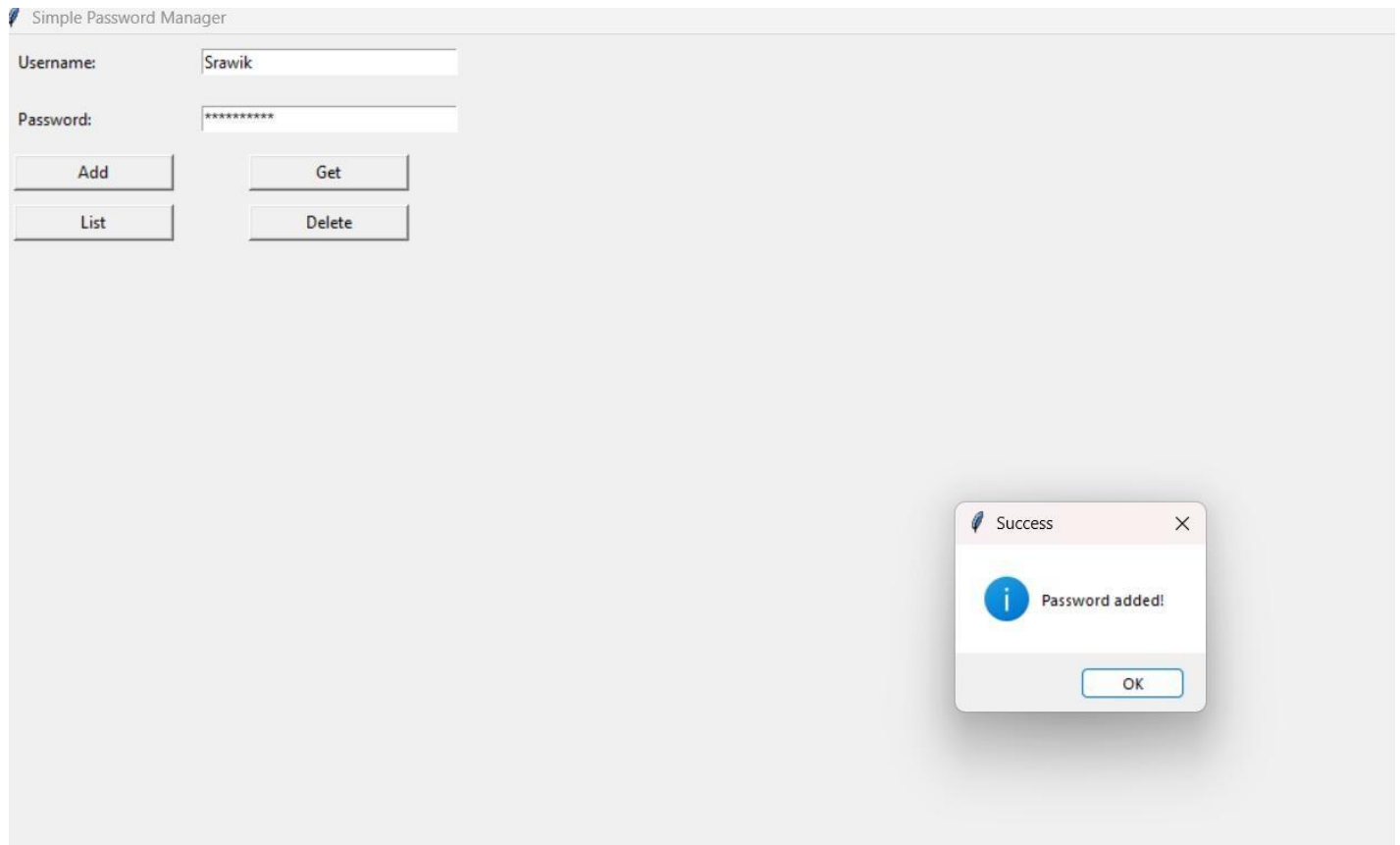
The image shows a graphical user interface for a 'Simple Password Manager'. The window has a title bar with a feather icon and the text 'Simple Password Manager'. Inside the window, there are two input fields: 'Username:' and 'Password:'. Below these fields are four buttons arranged in a 2x2 grid: 'Add', 'Get', 'List', and 'Delete'.

2. Adding username and password by clicking Add Button

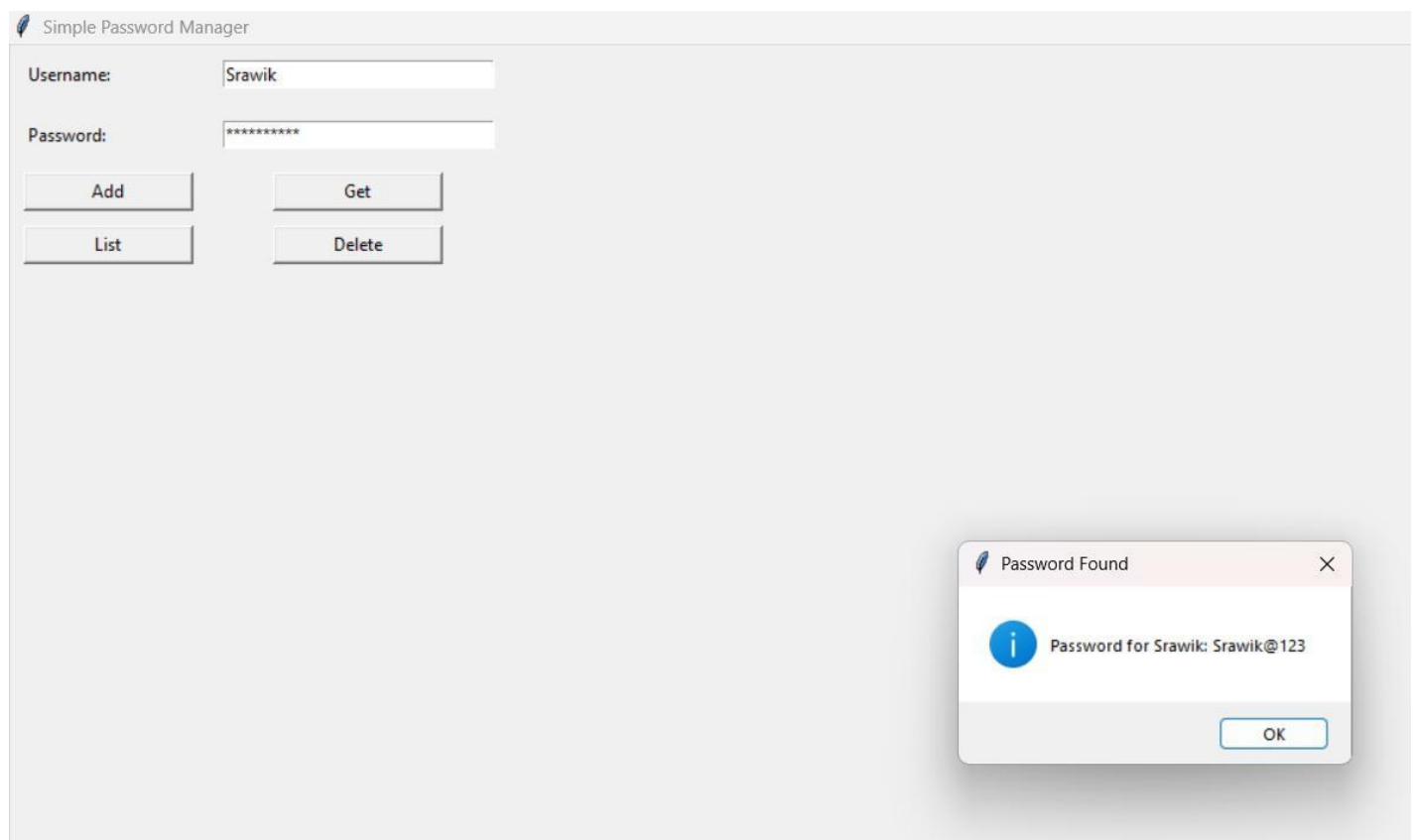


The image shows the same graphical user interface as in the first screenshot, but with data entered into the input fields. The 'Username:' field now contains the text 'SrawikDeekonda', and the 'Password:' field contains a series of asterisks '*****'. The four buttons ('Add', 'Get', 'List', 'Delete') remain in the same positions.

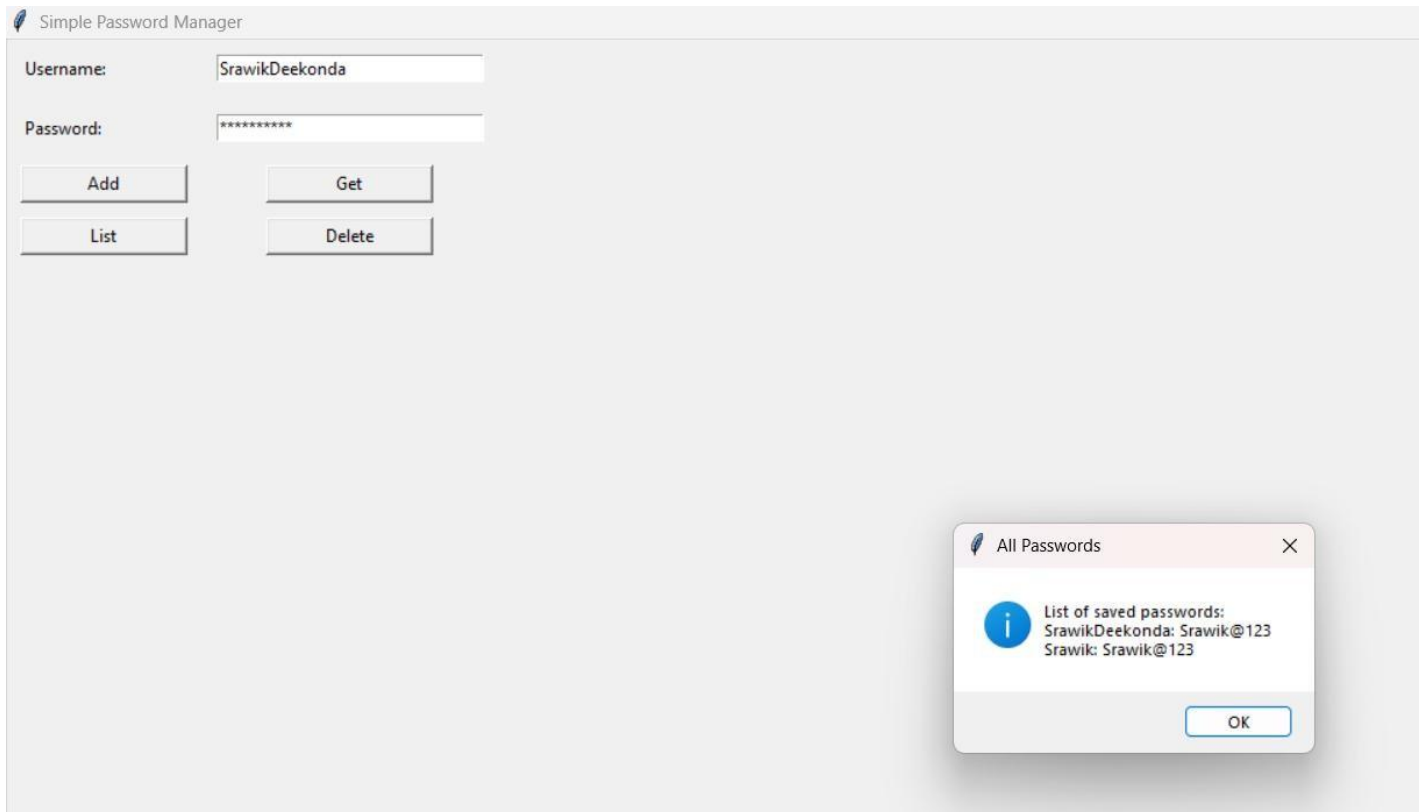
3. Adding another username and password and it say's Password added



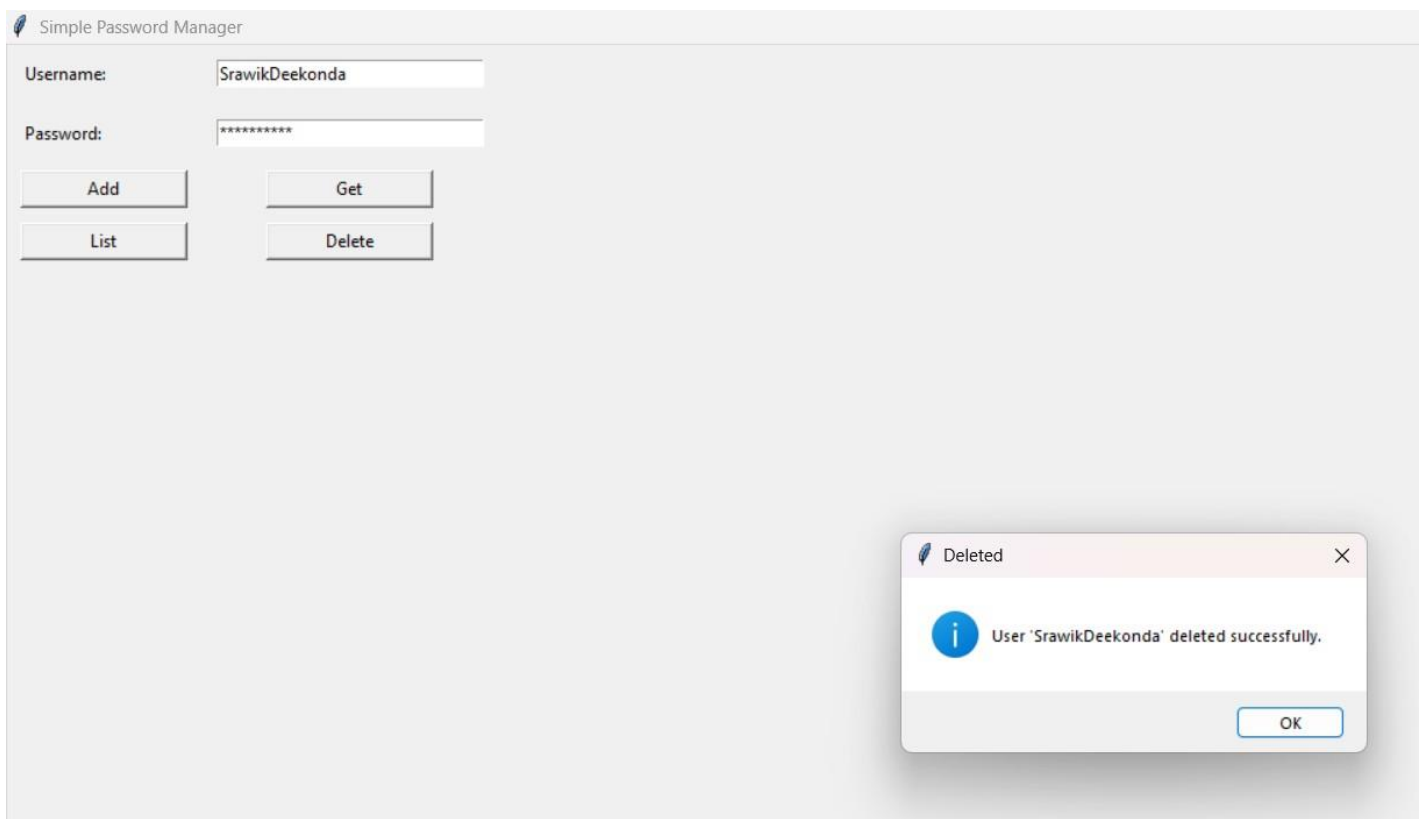
4. By typing Username and clicking Get Button it shows the password which we added



5. It shows the List of Saved Passwords which we added by Clicking List Button



6. By Clicking Delete button, the username will be deleted successfully



LEARNING OUTCOME

1. Understanding GUI Development

You gained practical knowledge in:

- Designing user-friendly interfaces using **Tkinter**
- Creating input fields, buttons, and message dialogs
- Organizing components using grid layout management
- Handling real-time user interactions through event-driven programming

2. Event-Driven Programming

You learned how:

- Functions can be triggered by user actions (like button clicks)
- The program flow can dynamically respond to inputs
- Different events (add, get, list, delete) are managed independently

3. File Handling in Python

You developed skills in:

- Reading and writing text files using Python
- Appending data (add), searching content (get), and modifying data (delete)
- Ensuring error handling using try-except blocks

4. Data Validation and Error Handling

You understood the importance of:

- Validating user input (e.g., checking if fields are empty)
- Handling exceptions like missing files or empty entries

- Providing user feedback via message boxes

5. Implementing Core Application Logic You implemented logic to:

- Add, retrieve, list, and delete credentials
- Work with data structures like dictionaries
- Maintain code modularity through clearly defined functions

6. Basics of Data Security Awareness

Even though plain text was used, this project introduced you to:

- The need for encrypting sensitive data like passwords
- Limitations of local storage
- Potential directions for security enhancement

7. Software Design Thinking

You were exposed to:

- Breaking down the application into components (GUI, Logic, Storage)
- Designing intuitive workflows for users
- Visualizing architecture using flowcharts

8. Communication & Documentation

By preparing flowcharts, diagrams, and presentations, you learned to:

- Document project flow clearly
- Communicate technical ideas through visuals
- Reflect on what worked and what can be improved

PROJECT IMPACT

- **User Empowerment:**

The application provides users with a simple tool to manage their credentials, reducing the need to remember multiple passwords.

- **Ease of Use:**

The intuitive Tkinter-based GUI makes it accessible even for users with minimal technical knowledge.

- **Local Storage Advantage:**

Since the data is stored locally, users have complete control over their passwords without relying on thirdparty services.

- **Educational Value:**

The project offers a practical learning experience in Python programming, GUI development, and file handling — making it a strong foundation for further software development pursuits.

FUTURE SCOPE

- **Password Encryption:**

Implement encryption (e.g., using Python's cryptography or hashlib modules) to protect stored passwords from unauthorized access.

- **Master Password System:**

Introduce a master password login to enhance security and prevent unauthorized access to stored credentials.

- **Migration to Secure Storage:**

Move from plain text storage to a more secure and structured system like SQLite or encrypted JSON files.

- **Cross-Platform Compatibility:**

Convert the script into a standalone application (using tools like PyInstaller or cx_Freeze) for Windows/Linux/macOS support.

- **Feature Expansion:**

Add features like password generation, expiry alerts, and automatic backups to enhance functionality.

- **Cloud Integration (Advanced):**

Incorporate cloud sync features to allow users to access their passwords securely from different devices.

- **User Authentication & Access Logs:**

Implement user-level authentication and maintain logs to monitor who accessed or modified data.

CONCLUSION

The Password Manager project stands as a practical implementation of core software development principles using Python and its built-in GUI library, Tkinter. The primary objective of this project was to create a userfriendly application that allows users to securely store, retrieve, view, and delete passwords associated with various usernames. Through this project, we were able to design and develop a working model that effectively fulfills these functionalities while maintaining simplicity and ease of use.

The graphical user interface built using Tkinter significantly enhances the usability of the application. It allows users to interact with the system without needing to use the command line or understand the underlying code. Each operation—Add, Get, List, and Delete—is connected to specific backend functions that handle file operations and user inputs. The integration of messagebox popups helps in providing immediate feedback to the user, thereby improving the interaction experience.

From a technical perspective, this project demonstrates the use of file handling for persistent storage. By saving credentials in a local text file, the system ensures that data is retained between sessions. Although this approach is suitable for beginners and small-scale applications, it also presents limitations, particularly regarding data security and scalability. Passwords are currently stored in plain text, making them vulnerable to unauthorized access if the file is exposed. This issue has been acknowledged and forms a crucial part of the future scope of this project.

Moreover, the implementation process highlighted key programming practices such as input validation, error handling, and modular code design. These aspects are fundamental to software development and were practically applied and reinforced throughout the project lifecycle.

In conclusion, the password manager application fulfills its core objective by providing a working solution for managing user credentials. It also serves as a strong foundation for more advanced versions in the future. The project not only demonstrates the developer's understanding of Python and GUI design but also raises awareness about the importance of data privacy and secure software practices. With enhancements such as encryption, database integration, and user authentication, this project holds the potential to evolve into a secure and full-featured password management system suitable for real-world use.

REFERENCES

1. Python Software Foundation. (2023). *Python 3 Documentation*. Retrieved from <https://docs.python.org/3/>
2. Tkinter Docs. (2023). *Tkinter GUI Programming*. Retrieved from <https://tkdocs.com/>
3. GeeksforGeeks. (2023). *Python GUI – tkinter*. Retrieved from <https://www.geeksforgeeks.org/pythongui-tkinter/>
4. W3Schools. (2023). *Python File Handling*. Retrieved from https://www.w3schools.com/python/python_file_handling.asp
5. TutorialsPoint. (2023). *Python GUI Programming using Tkinter*. Retrieved from https://www.tutorialspoint.com/python/python_gui_programming.htm
6. Real Python. (2023). *Build a GUI App Using Tkinter in Python*. Retrieved from <https://realpython.com/python-gui-tkinter/>
7. OWASP Foundation. (2022). *Password Storage Cheat Sheet*. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
8. Mozilla Developer Network. (2023). *Hashing and Encryption*. Retrieved from https://developer.mozilla.org/en-US/docs/Web/Security/Information_Security_Basics
9. PyPI. (2023). *cryptography Library for Python*. Retrieved from <https://pypi.org/project/cryptography/>
10. Stack Overflow. (2023). *Common Python GUI issues and solutions*. Retrieved from <https://stackoverflow.com/>